

Проверка в Editor (не на Build)

Стресс-тест - около 100 целей, большая частота выстрелов

Вариант 1

targets speed 5, period 0.2 (rate 5)

projectile speed 20, lifetime 4

Targets Amount - 200

ShootRate - 50

~100 targets, ~320 projectiles

Mode	Naive	Pool
FPS	38-42	38-44
Frame	~26ms	~24ms
Max instantiate calls	6.11 ms (6 calls)	0
Max destroy calls	2.58 ms (14 calls)	0

Вариант 2 (более быстрые объекты)

targets speed 10, period 0.1 (rate 10)

projectile speed 40, lifetime 1

Targets Amount - 200

ShootRate - 100

~100 targets, ~100 projectiles

Mode	Naive	Pool	Pool (before prewarm)
FPS	39	40-44	41
Frame	~25ms	~23ms	23
Max instantiate calls	6.11 ms (6 calls)	0	5.38ms (6 calls), stops at 92 frame
Max destroy calls	2.58 ms (14 calls)	0	0

Пояснения к компонентам для выставки теста:

Режим теста запускается при выборе `IsTargetsTestOn` и/или `IsProjectilesTestOn` на компоненте `TestSpawnAdjuster` объекта `PerformanceTest`

Объект `PerformanceTest`:

1. компонент `SpawnModeController` - выбор mode

2. компонент `TestSpawnAdjuster`

- `Targets`

параметры для `SpawnPoints` ("точка" спауна цели), которые при тесте будут выставляться в необходимом количестве в зоне спауна целей (существующие по умолчанию при тесте деактивируются). Частота выхода целей из `SpawnPoint` и скорость целей - в структуре `TestSpawnData`

- `Projectiles`

Скорость, время жизни снарядов в режиме теста

- `TestParameters`

`IsTargetsTestOn`, `IsProjectilesTestOn` - если отключены, то соответственно спаун целей / снарядов по умолчанию, то есть, как задано в параметрах объектов внутри `Gameplay` объекта

`TargetsAmount` (при `IsTargetsTestOn = true`) - сколько целей должно быть на сцене при тесте (так как часть целей уничтожается, нужно подбирать больше, чем желаемое число)

`ShootRate` (при `IsProjectilesTestOn = true`) - частота выстрелов (исходя из неё, скорости снарядов и расстояния от турели до зоны целей по оси `z`) рассчитывается размер `prewarm` для пула снарядов

Часть 2

Что сделано

1. Сделаны MovableSystem и LifetimedSystem и интерфейсы IMovable, ILifetimed для контроля в этих системах

В Target и Projectile теперь сами не вызывают Update(), имплементируют интерфейсы и контролируются системами. На остальных игровых объектах уже были корутины

2. В FPSCounter (переименован из PerformanceCounter) - замена проверок каждый кадр на проверку в корутине плюс удалён отдельный таймер для расчёта времени кадра

В остальных системах, как на сцене, так и в UI, уже реализовано через корутины или events

3. TargetsService и ProjectilesService сделаны обычными C#-классами, так же из них убрана информация с именами заголовков, нужными для отображения в UI

4. Разделены Canvas для динамических и статических объектов

Разница

По времени:

Медианное время UpdateScene снизилось на 0.79 ms

Время на BehaviourUpdate:

В предыдущей версии - 0.35..1.7 ms

В нынешней версии - 0.44..0.78 ms

То есть, время более стабильное и пиковое значение намного меньше

Общая частота кадров стала выше

По архитектуре:

Пришлось незначительно изменить код классов, которые до этого сами вызывали Update(). С другой стороны, появилось обобщение для объектов, которые движутся, у которых есть время жизни, что может в будущем упростить само продумывание новых классов с подобным функционалом.

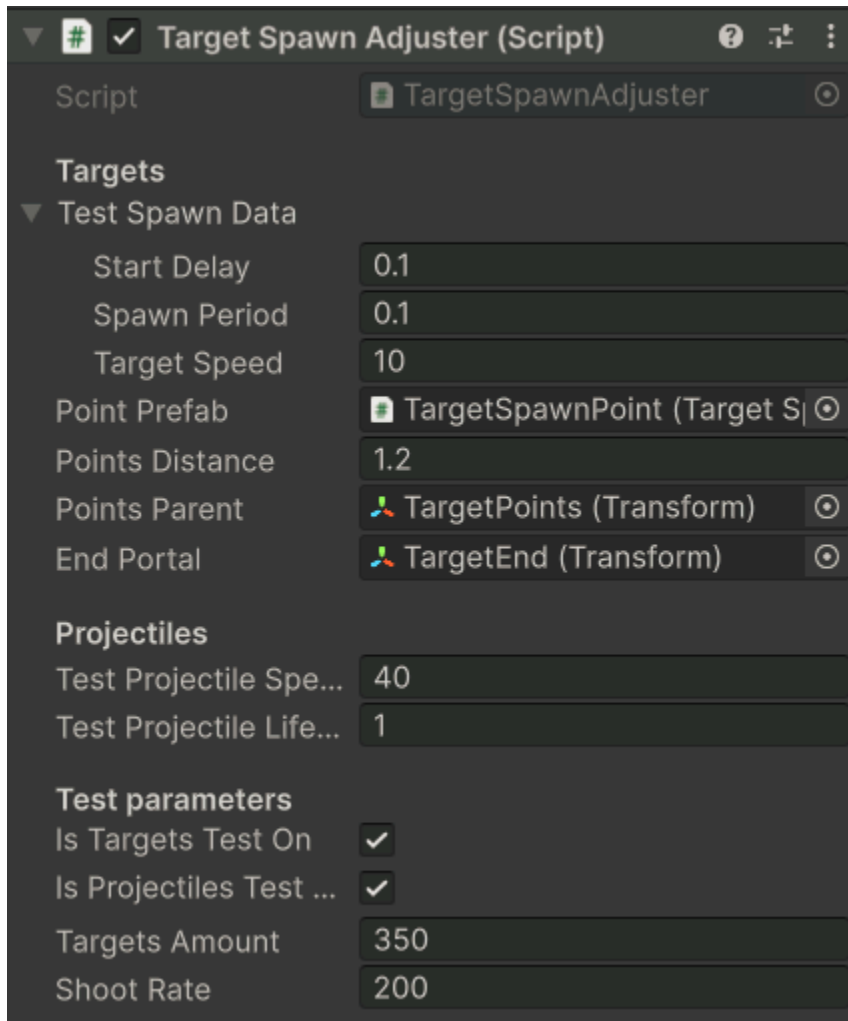
Увеличилось количество полей в GameInitializer, для инициализации в бывшие MonoBehaviour

Архитектура и понимание кода скорее немного усложнились

Часть 3

Условия стресс-теста

- Тестирование в Editor (не на build-версии)
- Режим Pool
- Запуск без prewarm, но с ожиданием ~10 с до момента, когда PoolSize чуть больше, чем OnScene для обоих пулов
- Разрешение экрана 960*540
- Настройки TargetSpawnAdjuster на объекте PerformanceTest:



Анализ до внесения изменений

1. Основные нагрузки возникают в моменты spawn/inspawn в пул целей и снарядов:

- в корутинах спауна в SpawnPoint; в тесте точки синхронизированы, поэтому в профайлере видны периодические пики

max:

10 calls

время 4.47 ms (~3ms)

GC.Alloc ~3KB

- в корутинах пушек Turret (разная частота выстрелов); периодических пиков нет, т.к. не синхронизированы

max (редко):

1.16 ms

GC.Alloc 0.7KB

2. Небольшие аллокации:

- в Update() LifetimedSystem при уничтожении снарядов - max 420B

- в FixedUpdate() в OnTriggerEnter() при уничтожении сталкивающихся объектов

max ~2.5KB (больше от Projectile.OnTriggerEnter(), а от Target.OnTriggerEnter() max 424B

т.к. небольшое количество целей долетает одновременно до зоны уничтожения целей)

3. Нагрузки по времени, но без аллокаций:

MovablesSystem

.Update max 0.77ms ~0.2

.FixedUpdate max 1.55ms ~0.55

4. UI

UGUI.Rendering.UpdateBatches

max 1.40ms, mean 0.51ms

PostLateUpdate.PlayerUpdateCanvases

max ~1ms

В большей степени за счёт TMP.GenerateText ~0.73

Что не подходит по оптимизации:

1. В корутине спауна целей

- В коде в циклах корутин нет `new WaitForSeconds`

- Вариант общего менеджера-корутины спауна целей и, аналогично - для пушек турели не подходит. Пушки уже в тесте стреляют с разной частотой. Цели в тесте спаунятся синхронно, но для свободы лучше оставить возможность менять частоту `SpawnPoint` индивидуально.

2. UI

В коде нет изменений UI в каждом кадре - всё по событиям при изменении или раз в период. Разделение на статический и динамический уже сделано на предыдущем этапе

3. Аллокации при создании/уничтожении объектов

Частично вызваны созданием строк в UI (`ObjectSpawnInfoPresenter` - счётчики объектов). `TMP_Text` предполагает использование строк, поэтому данной аллокации не избежать

4. Нагрузки в `MovablesSystem` и `LifetimedSystem`

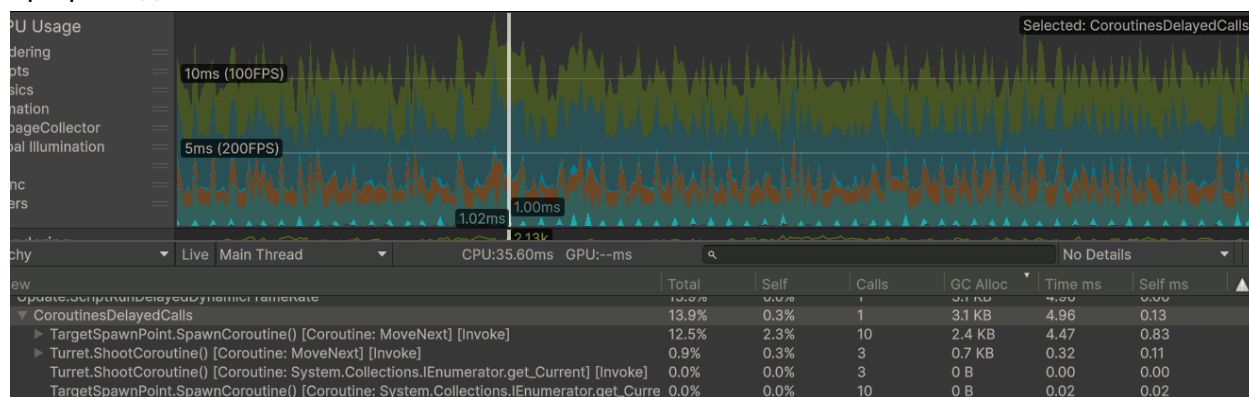
Данные системы сами являются способом оптимизации: не вижу вариантов их как-то ещё оптимизировать

5. `string concatenation`, `Debug.Log()`, LINQ, `foreach`, частые новые массивы/списки, `boxing/unboxing` в коде не замечены

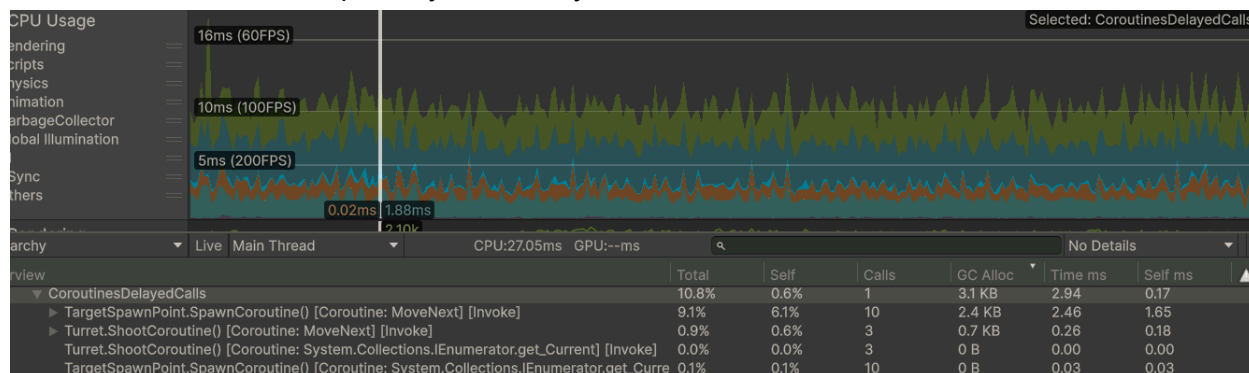
Оптимизация

1. Основное время в пиках корутин спауна целей - `GameObject.Activate` (особенно `Animator.Rebind`), поэтому убран `SetActive()` при спауне/анспауне. Вместо него - интерфейс `IPoolable` с методом `Activate(bool)`, который делает неактивными только некоторые компоненты на цели и на снаряде

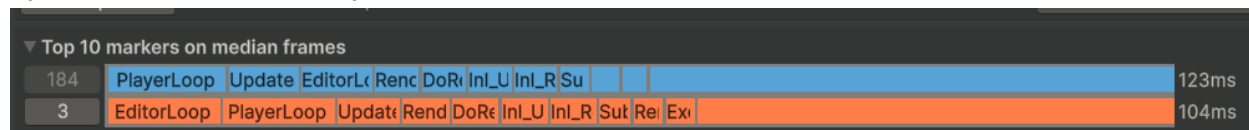
Профайл до изменений:



После изменений пики при спауне целей ушли:



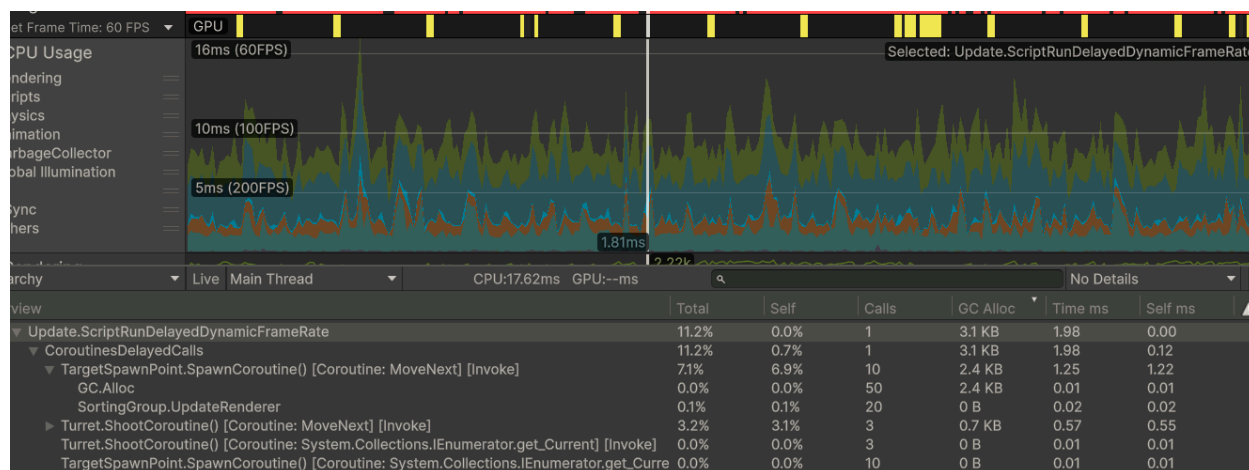
Суммарное время кадра уменьшилось:



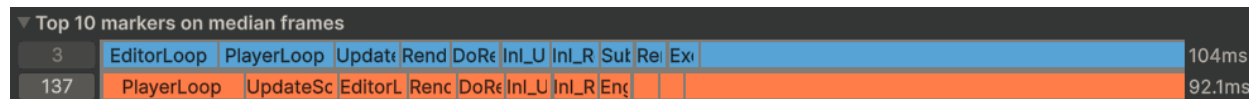
2. Остались затраты по времени при спауне целей и снарядов, вызванные Transform.SetParent - max суммарно по кадру для всех объектов 1.4ms

Поэтому SetParent() при спауне/анспауне убран. PoolTransform удалены, оставлены ParentTransform для одновременно активных и неактивных снарядов и аналогично для целей. SetParent() вызывается для объекта только один раз - при первом создании в пуле.

После изменений исчезли частые вызовы Transform.SetParent



Суммарное время кадра немного уменьшилось



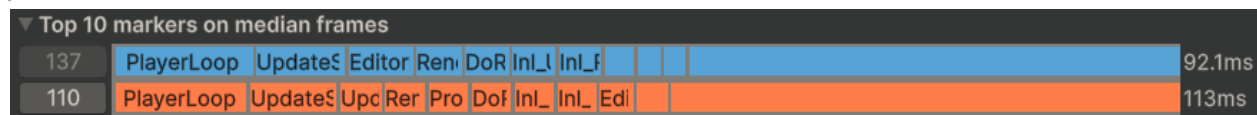
3. Для цели изменена настройка в Animator на CullCompletely. Каких-либо улучшений не замечено

4. Аллокации при спауне/анспауне целей и снарядов

Частично обусловлены подпиской/отпиской в

TargetSpawnManager/ProjectileSpawnManager в методах Spawn() и Unspawn(). Для подписки на Unspawn только один раз при первом Instantiate объекта: в IPool введён event Action<T> OnNewInstantiated, созданы похожие ProjectilesService и TargetsService, которые подписывают новые созданные объекты на Unspawn и отписывают их все в Dispose().

Суммарное время кадра немного увеличилось (но PlayerLoop по медианному кадру не увеличился)



При анспауне аллокации уменьшились: max 1.1KB для Target, 0B для Projectile против 2.5KB и 424KB соответственно

Максимальные аллокации при спауне уменьшились (1.1KB для Target и 342B для Projectile против 2.4KB и 0.7KB соответственно на профайле выше)

